Computer Networks Lab Record

March 2, 2026

Contents

client.c . . . 2
server.c . . . 3
Output . . . 4
matrix_client.c . . . 5
matrix_server.c . . . 6
Output . . . 7
client.c . . . 8
server.c . . . 9
Output . . . 9
client.c . . . 10
server.c . . . 10
Output . . . 11
client.c . . . 12
server.c . . . 13
Output . . . 13
client.c . . . 14
server.c . . . 15
Output . . . 15
swarq.c . . . 16
Output . . . 17
gnarq.c . . . 18
Output . . . 19
srarq.c . . . 20
Output . . . 21
client.c . . . 22
server.c . . . 23
Output . . . 24
client.c . . . 25
server.c . . . 26
Output . . . 26
lsra.c . . . 27
Output . . . 28
client.c . . . 29
server.c . . . 30
test.txt . . . 31
Output . . . 31
lba.c . . . 32
Output . . . 33

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main() {
    int sockfd;
    struct sockaddr_in server;
    char buffer[1024];

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    server.sin_family = AF_INET;
    server.sin_port = htons(9000);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(sockfd, (struct sockaddr *)&server, sizeof(server));

    printf("Enter string: ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\n")] = 0;

    send(sockfd, buffer, strlen(buffer), 0);

    int n = recv(sockfd, buffer, sizeof(buffer), 0);
    buffer[n] = '\0';

    printf("Reversed string from server: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

void reverse_string(char *str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char tmp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = tmp;
    }
}

int main() {
    int sockfd, connfd;
    struct sockaddr_in server, client;
    char buffer[1024];
    socklen_t len;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    server.sin_family = AF_INET;
    server.sin_port = htons(9000);
    server.sin_addr.s_addr = htonl(INADDR_ANY);

    bind(sockfd, (struct sockaddr *)&server, sizeof(server));
    listen(sockfd, 5);

    printf("Server listening on port 9000...\n");

    len = sizeof(client);
    connfd = accept(sockfd, (struct sockaddr *)&client, &len);

    int n = recv(connfd, buffer, sizeof(buffer), 0);
    buffer[n] = '\0';

    printf("Received from client: %s\n", buffer);

    reverse_string(buffer);

    send(connfd, buffer, strlen(buffer), 0);

    close(connfd);
    close(sockfd);
    return 0;
}
```

Output

```
server.c
Server listening on port 9000...
Received from client: Malayalam

client.c
Enter string: Malayalam
Reversed string from server: malayalaM
```

matrix_client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define SIZE 3
int main() {
    int sockfd;
    struct sockaddr_in serverAddr;
    int A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];
    socklen_t addr_size;

    printf("Enter Matrix A (3x3):\n");
    for (int i = 0; i < SIZE; i++)
        for (int j = 0; j < SIZE; j++)
            scanf("%d", &A[i][j]);

    printf("Enter Matrix B (3x3):\n");
    for (int i = 0; i < SIZE; i++)
        for (int j = 0; j < SIZE; j++)
            scanf("%d", &B[i][j]);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    addr_size = sizeof(serverAddr);

    sendto(sockfd, A, sizeof(A), 0, (struct sockaddr *)&serverAddr,
           addr_size);
    sendto(sockfd, B, sizeof(B), 0, (struct sockaddr *)&serverAddr,
           addr_size);

    recvfrom(sockfd, C, sizeof(C), 0, NULL, NULL);

    printf("\nResult Matrix (A times B):\n");
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    close(sockfd);
    return 0;
}
```

matrix_server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SIZE 3

int main() {
    int sockfd;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addr_size;
    int A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    bind(sockfd, (struct sockaddr *)&serverAddr, sizeof(serverAddr));

    addr_size = sizeof(clientAddr);

    printf("Server waiting for matrices...\n");

    recvfrom(sockfd, A, sizeof(A), 0, (struct sockaddr *)&clientAddr,
             &addr_size);

    recvfrom(sockfd, B, sizeof(B), 0, (struct sockaddr *)&clientAddr,
             &addr_size);

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            C[i][j] = 0;
            for (int k = 0; k < SIZE; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    sendto(sockfd, C, sizeof(C), 0, (struct sockaddr *)&clientAddr,
           addr_size);

    printf("Matrix multiplication completed and result sent.\n");

    close(sockfd);
    return 0;
}
```

Output

```
matrix_server.c
Server waiting for matrices...
Matrix multiplication completed and result sent.

matrix_client.c
Enter Matrix A (3x3):
1 2 3
4 5 6
7 8 9
Enter Matrix B (3x3):
9 8 7
6 5 4
3 2 1

Result Matrix (A × B):
30 24 18
84 69 54
138 114 90
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, num, sum;
    struct sockaddr_in server;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(sockfd, (struct sockaddr *)&server, sizeof(server));
    printf("Enter a number: ");
    scanf("%d", &num);
    write(sockfd, &num, sizeof(num));
    read(sockfd, &sum, sizeof(sum));
    printf("Sum of digits: %d\n", sum);
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    int sockfd, newsockfd, num, sum = 0;
    struct sockaddr_in server, client;
    socklen_t len = sizeof(client);
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = htonl(INADDR_ANY);
    bind(sockfd, (struct sockaddr *)&server, sizeof(server));
    listen(sockfd, 5);
    printf("Server is listening on port 8080...\n");
    newsockfd = accept(sockfd, (struct sockaddr *)&client, &len);
    read(newsockfd, &num, sizeof(num));
    while (num > 0) {
        sum += num % 10;
        num /= 10;
    }
    write(newsockfd, &sum, sizeof(sum));
    read(newsockfd, &sum, sizeof(sum));
    printf("result calculated\n");
    close(newsockfd);
    close(sockfd);
    return 0;
}
```

Output

```
server.c
Server is listening on port 8080...
result calculated

client.c
Enter a number: 17
Sum of digits: 8
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, num, sum;
    struct sockaddr_in server;
    socklen_t len = sizeof(server);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    printf("Enter a number: ");
    scanf("%d", &num);
    sendto(sockfd, &num, sizeof(num), 0, (struct sockaddr *)&server, len);
    recvfrom(sockfd, &sum, sizeof(sum), 0, (struct sockaddr *)&server, &len);
    printf("Sum of digits: %d\n", sum);
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, num, sum = 0;
    struct sockaddr_in server, client;
    socklen_t len = sizeof(client);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (struct sockaddr *)&server, sizeof(server));
    printf("Server is listening on port 8080...\n");
    recvfrom(sockfd, &num, sizeof(num), 0, (struct sockaddr *)&client, &len);
    while (num > 0) {
        sum += num % 10;
        num /= 10;
    }
    sendto(sockfd, &sum, sizeof(sum), 0, (struct sockaddr *)&client, len);
    printf("result calculated\n");
    close(sockfd);
    return 0;
}
```

Output

```
server.c
Server is listening on port 8080...
result calculated

client.c
Enter a number: 17
Sum of digits: 8
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, n, key, pos;
    int arr[50];
    struct sockaddr_in server;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    connect(sockfd, (struct sockaddr *)&server, sizeof(server));
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter the key to search: ");
    scanf("%d", &key);
    write(sockfd, &n, sizeof(n));
    write(sockfd, arr, n * sizeof(int));
    write(sockfd, &key, sizeof(key));
    read(sockfd, &pos, sizeof(pos));
    if (pos >= 0)
        printf("Element found at position: %d\n", pos);
    else
        printf("Element not found in the array.\n");
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, newsockfd, n, key, pos;
    int arr[50];
    struct sockaddr_in server, client;
    socklen_t len = sizeof(client);
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    bind(sockfd, (struct sockaddr *)&server, sizeof(server));
    listen(sockfd, 5);
    printf("Server is listening on port 8080...\n");
    newsockfd = accept(sockfd, (struct sockaddr *)&client, &len);
    read(newsockfd, &n, sizeof(n));
    read(newsockfd, arr, n * sizeof(int));
    read(newsockfd, &key, sizeof(key));
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            pos = i;
            break;
        } else {
            pos = -1;
        }
    }
    write(newsockfd, &pos, sizeof(pos));
    printf("result calculated\n");
    close(newsockfd);
    close(sockfd);
    return 0;
}
```

Output

```
server.c
Server is listening on port 8080...
result calculated

client.c
Enter the number of elements: 5
Enter the elements: 1
2
3
4
5
Enter the key to search: 2
Element found at position: 1
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, n, key, pos;
    int arr[50];
    struct sockaddr_in server;
    socklen_t len = sizeof(server);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    printf("Enter the elements of the array: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter the key to search: ");
    scanf("%d", &key);
    sendto(sockfd, &n, sizeof(n), 0, (struct sockaddr *)&server, len);
    sendto(sockfd, arr, sizeof(arr), 0, (struct sockaddr *)&server, len);
    sendto(sockfd, &key, sizeof(key), 0, (struct sockaddr *)&server, len);
    recvfrom(sockfd, &pos, sizeof(pos), 0, (struct sockaddr *)&server, &len);
    if (pos >= 0) {
        printf("Element found at position: %d\n", pos);
    } else {
        printf("Element not found in the array.\n");
    }
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    int sockfd, n, key, pos = -1;
    int arr[50];
    struct sockaddr_in server, client;
    socklen_t len = sizeof(client);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(8080);
    server.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (struct sockaddr *)&server, sizeof(server));
    printf("Server is listening on port 8080...\n");
    recvfrom(sockfd, &n, sizeof(n), 0, (struct sockaddr *)&client, &len);
    recvfrom(sockfd, arr, sizeof(arr), 0, (struct sockaddr *)&client, &len);
    recvfrom(sockfd, &key, sizeof(key), 0, (struct sockaddr *)&client, &len);
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            pos = i;
            break;
        }
    }
    sendto(sockfd, &pos, sizeof(pos), 0, (struct sockaddr *)&client, len);
    printf("result calculated\n");
    close(sockfd);
    return 0;
}
```

Output

```
server.c
Server is listening on port 8080...
result calculated

client.c
Enter the number of elements: 5
Enter the elements: 1
2
3
4
5
Enter the key to search: 2
Element found at position: 1
```

swarq.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int frame = 0;
    int ack;
    int choice;

    while (frame < 10) {
        printf("\nSender: Sending frame %d", frame);

        ack = rand() % 2;

        if (ack == 1) {
            printf("\nReceiver: ACK received for frame %d", frame);
            frame++;
        } else {
            printf("\nReceiver: ACK lost for frame %d", frame);
            printf("\nSender: Timeout! Resending frame %d", frame);
        }

        printf("\n-----------------------------------");
    }

    printf("\nAll frames transmitted successfully.\n");
    return 0;
}
```

Output

```
Sender: Sending frame 0
Receiver: ACK received for frame 0
-----------------------------------
Sender: Sending frame 1
Receiver: ACK lost for frame 1
Sender: Timeout! Resending frame 1
-----------------------------------
Sender: Sending frame 1
Receiver: ACK received for frame 1
-----------------------------------
Sender: Sending frame 2
Receiver: ACK received for frame 2
-----------------------------------
Sender: Sending frame 3
Receiver: ACK received for frame 3
-----------------------------------
Sender: Sending frame 4
Receiver: ACK received for frame 4
-----------------------------------
Sender: Sending frame 5
Receiver: ACK lost for frame 5
Sender: Timeout! Resending frame 5
-----------------------------------
Sender: Sending frame 5
Receiver: ACK lost for frame 5
Sender: Timeout! Resending frame 5
-----------------------------------
Sender: Sending frame 5
Receiver: ACK received for frame 5
-----------------------------------
Sender: Sending frame 6
Receiver: ACK received for frame 6
-----------------------------------
Sender: Sending frame 7
Receiver: ACK lost for frame 7
Sender: Timeout! Resending frame 7
-----------------------------------
Sender: Sending frame 7
Receiver: ACK received for frame 7
-----------------------------------
Sender: Sending frame 8
Receiver: ACK lost for frame 8
Sender: Timeout! Resending frame 8
-----------------------------------
Sender: Sending frame 8
Receiver: ACK received for frame 8
-----------------------------------
Sender: Sending frame 9
Receiver: ACK received for frame 9
-----------------------------------
All frames transmitted successfully.
```

gnarq.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_FRAMES 10
#define WINDOW_SIZE 4

int isFrameLost() {
    return rand() % 5 == 0;
}

int main() {
    int totalFrames = MAX_FRAMES;
    int windowSize = WINDOW_SIZE;
    int base = 0;
    int nextSeqNum = 0;
    int ack;

    srand(time(0));

    printf("Go-Back-N ARQ Simulation\n");
    printf("Total frames: %d, Window size: %d\n\n", totalFrames, windowSize);

    while (base < totalFrames) {

        while (nextSeqNum < base + windowSize && nextSeqNum < totalFrames) {
            printf("Frame %d sent successfully.\n", nextSeqNum);
            nextSeqNum++;
        }

        ack = base;
        while (ack < nextSeqNum) {
            if (isFrameLost()) {
                printf("ACK for Frame %d lost! Retransmitting from Frame %d\n",
                    ack, ack);
                nextSeqNum = ack;
                break;
            } else {
                printf("ACK received for Frame %d\n", ack);
                ack++;
            }
        }
        base = ack;
        printf("\n");
    }

    printf("All frames transmitted successfully!\n");
    return 0;
}
```

Output

```
Go-Back-N ARQ Simulation
Total frames: 10, Window size: 4

Frame 0 sent successfully.
Frame 1 sent successfully.
Frame 2 sent successfully.
Frame 3 sent successfully.
ACK received for Frame 0
ACK received for Frame 1
ACK received for Frame 2
ACK received for Frame 3

Frame 4 sent successfully.
Frame 5 sent successfully.
Frame 6 sent successfully.
Frame 7 sent successfully.
ACK for Frame 4 lost! Retransmitting from Frame 4

Frame 4 sent successfully.
Frame 5 sent successfully.
Frame 6 sent successfully.
Frame 7 sent successfully.
ACK received for Frame 4
ACK received for Frame 5
ACK for Frame 6 lost! Retransmitting from Frame 6

Frame 6 sent successfully.
Frame 7 sent successfully.
Frame 8 sent successfully.
Frame 9 sent successfully.
ACK received for Frame 6
ACK received for Frame 7
ACK received for Frame 8
ACK received for Frame 9

All frames transmitted successfully!
```

srarq.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, windowSize;
    int i, j;
    int ack[50];

    printf("Enter number of frames: ");
    scanf("%d", &n);

    printf("Enter window size: ");
    scanf("%d", &windowSize);

    for (i = 0; i < n; i++)
        ack[i] = 0;

    printf("\n--- Selective Repeat ARQ Simulation ---\n");

    for (i = 0; i < n; i += windowSize) {

        for (j = i; j < i + windowSize && j < n; j++) {
            printf("Frame %d sent\n", j);
        }

        for (j = i; j < i + windowSize && j < n; j++) {
            if (ack[j] == 0) {
                printf("Did receiver get frame %d correctly? (1=Yes, 0=No): ", j);
                scanf("%d", &ack[j]);

                if (ack[j] == 0) {
                    printf("Frame %d lost! Retransmitting frame %d\n", j, j);
                    printf("Frame %d resent successfully\n", j);
                    ack[j] = 1;
                }
            }
        }
        printf("\n");
    }

    printf("All frames received successfully!\n");
    return 0;
}
```

Output

```
Enter number of frames: 5
Enter window size: 3

--- Selective Repeat ARQ Simulation ---
Frame 0 sent
Frame 1 sent
Frame 2 sent
Did receiver get frame 0 correctly? (1=Yes, 0=No): 1
Did receiver get frame 1 correctly? (1=Yes, 0=No): 0
Frame 1 lost! Retransmitting frame 1
Frame 1 resent successfully
Did receiver get frame 2 correctly? (1=Yes, 0=No): 1

Frame 3 sent
Frame 4 sent
Did receiver get frame 3 correctly? (1=Yes, 0=No): 1
Did receiver get frame 4 correctly? (1=Yes, 0=No): 1

All frames received successfully!
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <unistd.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sock;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];
    fd_set readfds;
    sock = socket(AF_INET, SOCK_STREAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);
    connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to chat server...\n");
    while (1) {
        FD_ZERO(&readfds);
        FD_SET(0, &readfds);
        FD_SET(sock, &readfds);

        select(sock + 1, &readfds, NULL, NULL, NULL);

        if (FD_ISSET(0, &readfds)) {
            fgets(buffer, BUFFER_SIZE, stdin);
            send(sock, buffer, strlen(buffer), 0);
        }

        if (FD_ISSET(sock, &readfds)) {
            int valread = read(sock, buffer, BUFFER_SIZE);
            if (valread > 0) {
                buffer[valread] = '\0';
                printf("%s", buffer);
            }
        }
    }
    close(sock);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <unistd.h>
#define PORT 8080
#define MAX_CLIENTS 10
#define BUFFER_SIZE 1024
int main() {
    int server_fd, new_socket, client_socket[MAX_CLIENTS];
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];
    fd_set readfds;
    int max_sd, sd, activity;

    for (int i = 0; i < MAX_CLIENTS; i++)
        client_socket[i] = 0;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    printf("Chat server started on port %d...\n", PORT);
    while (1) {
        FD_ZERO(&readfds);
        FD_SET(server_fd, &readfds);
        max_sd = server_fd;
        for (int i = 0; i < MAX_CLIENTS; i++) {
            sd = client_socket[i];
            if (sd > 0)
                FD_SET(sd, &readfds);
            if (sd > max_sd)
                max_sd = sd;
        }
        activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);

        if (FD_ISSET(server_fd, &readfds)) {
            new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t
               *)&addrlen);
            printf("New client connected: %d\n", new_socket);
            for (int i = 0; i < MAX_CLIENTS; i++) {
                if (client_socket[i] == 0) {
                    client_socket[i] = new_socket;
                    break;
                }
            }
        }

        for (int i = 0; i < MAX_CLIENTS; i++) {
            sd = client_socket[i];

            if (FD_ISSET(sd, &readfds)) {
                int valread = read(sd, buffer, BUFFER_SIZE);
                if (valread == 0) {
                    close(sd);
                    client_socket[i] = 0;
                    printf("Client disconnected\n");
                } else {
```

```
                    buffer[valread] = '\0';
                    printf("Client %d: %s", sd, buffer);

                    for (int j = 0; j < MAX_CLIENTS; j++) {
                        if (client_socket[j] != 0 && client_socket[j] != sd) {
                            send(client_socket[j], buffer, strlen(buffer), 0);
                        }
                    }
                }
            }
        }
    }
    return 0;
}
```

Output

```
server.c
Chat server started on port 8080...
New client connected: 4
Client 4: Hello world!
Client 4: welcome

client.c
Connected to chat server...
Hello world!
welcome
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr;
    socklen_t addr_len = sizeof(server_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    sendto(sockfd, "Time Request", strlen("Time Request"), 0,
           (struct sockaddr *)&server_addr, addr_len);

    recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&server_addr,
             &addr_len);
    printf("Time from server: %s\n", buffer);
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    time_t now;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("UDP Concurrent Server running on port %d...\n", PORT);
    while (1) {

        recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
                 (struct sockaddr *)&client_addr, &addr_len);
        printf("Request received from client\n");

        now = time(NULL);
        snprintf(buffer, BUFFER_SIZE, "%s", ctime(&now));

        sendto(sockfd, buffer, strlen(buffer), 0,
               (struct sockaddr *)&client_addr, addr_len);
    }
    close(sockfd);
    return 0;
}
```

Output

```
server.c
UDP Concurrent Server running on port 8080...
Request received from client

client.c
Time from server: Sun Mar  1 20:03:59 2026
```

lsra.c

```c
#include <stdio.h>
#define MAX 10
#define INF 9999
int main() {
    int n, i, j, source;
    int cost[MAX][MAX], dist[MAX], visited[MAX];

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter cost matrix (9999 for no direct link):\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    printf("Enter source node: ");
    scanf("%d", &source);

    for (i = 0; i < n; i++) {
        dist[i] = cost[source][i];
        visited[i] = 0;
    }
    visited[source] = 1;
    dist[source] = 0;

    for (i = 0; i < n - 1; i++) {
        int min = INF, u = -1;
        for (j = 0; j < n; j++) {
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        }

        visited[u] = 1;
        for (j = 0; j < n; j++) {
            if (!visited[j] && dist[u] + cost[u][j] < dist[j]) {
                dist[j] = dist[u] + cost[u][j];
            }
        }
    }

    printf("\nShortest paths from node %d:\n", source);
    for (i = 0; i < n; i++) {
        printf("To node %d -> Cost = %d\n", i, dist[i]);
    }
    return 0;
}
```

Output

```
Enter number of nodes: 2
Enter cost matrix (9999 for no direct link):
2
3
5
0
Enter source node: 2

Shortest paths from node 2:
To node 0 -> Cost = 0
To node 1 -> Cost = 0
```

client.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char filename[BUFFER_SIZE];
    char buffer[BUFFER_SIZE];
    int n;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);

    connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("Enter filename: ");
    scanf("%s", filename);

    send(sockfd, filename, strlen(filename), 0);

    printf("\n--- Server Response ---\n");
    while ((n = read(sockfd, buffer, BUFFER_SIZE)) > 0) {
        buffer[n] = '\0';
        printf("%s", buffer);
    }
    close(sockfd);
    return 0;
}
```

server.c

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char filename[BUFFER_SIZE];
    char buffer[BUFFER_SIZE];
    pid_t pid;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    listen(server_fd, 5);
    printf("Concurrent File Server running on port %d...\n", PORT);
    while (1) {

        client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len);

        pid = fork();
        if (pid == 0) {
            close(server_fd);

            int n = read(client_fd, filename, BUFFER_SIZE - 1);
            filename[n] = '\0';
            FILE *fp = fopen(filename, "r");
            pid_t server_pid = getpid();
            if (fp == NULL) {
                snprintf(buffer, BUFFER_SIZE,
                         "Server PID: %d\nFile not found\n", server_pid);
                send(client_fd, buffer, strlen(buffer), 0);
            } else {
                snprintf(buffer, BUFFER_SIZE,
                         "Server PID: %d\nFile content:\n", server_pid);
                send(client_fd, buffer, strlen(buffer), 0);
                while (fgets(buffer, BUFFER_SIZE, fp) != NULL) {
                    send(client_fd, buffer, strlen(buffer), 0);
                }
                fclose(fp);
            }
            close(client_fd);
            exit(0);
        }

        close(client_fd);
    }

    return 0;
}
```

test.txt

```
Hello, this is a test file!
```

Output

```
server.c
Concurrent File Server running on port 8080...

client.c
Enter filename: test.txt

--- Server Response ---
Server PID: 53572
File content:
Hello, this is a test file!
```

lba.c

```c
#include <stdio.h>
int main() {
    int bucket_size, output_rate, n;
    int packet_size;
    int bucket_content = 0;
    int i;
    printf("Enter Bucket Size: ");
    scanf("%d", &bucket_size);
    printf("Enter Output Rate: ");
    scanf("%d", &output_rate);
    printf("Enter Number of Packets: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("\nEnter size of packet %d: ", i);
        scanf("%d", &packet_size);
        printf("Packet %d size = %d\n", i, packet_size);
        if (packet_size > bucket_size) {
            printf("Packet Dropped (Too Large)\n");
        } else if (bucket_content + packet_size > bucket_size) {
            printf("Packet Dropped (Bucket Overflow)\n");
        } else {
            bucket_content += packet_size;
            printf("Packet Accepted\n");
        }
        printf("Bucket Content before leak: %d\n", bucket_content);

        if (bucket_content > output_rate)
            bucket_content -= output_rate;
        else
            bucket_content = 0;

        printf("Bucket Content after leak: %d\n", bucket_content);
    }

    return 0;
}
```

Output

```
Enter Bucket Size: 10
Enter Output Rate: 4
Enter Number of Packets: 4

Enter size of packet 1: 6
Packet 1 size = 6
Packet Accepted
Bucket Content before leak: 6
Bucket Content after leak: 2

Enter size of packet 2: 3
Packet 2 size = 3
Packet Accepted
Bucket Content before leak: 5
Bucket Content after leak: 1

Enter size of packet 3: 7
Packet 3 size = 7
Packet Accepted
Bucket Content before leak: 8
Bucket Content after leak: 4

Enter size of packet 4: 2
Packet 4 size = 2
Packet Accepted
Bucket Content before leak: 6
Bucket Content after leak: 2
```